



Third Year Engineering

21BTCS504-Operating System

Class - T.Y. (SEM-I)

Unit - II ***PROCESS MANAGEMENT***

AY 2024-2025 SEM-I





Unit-II

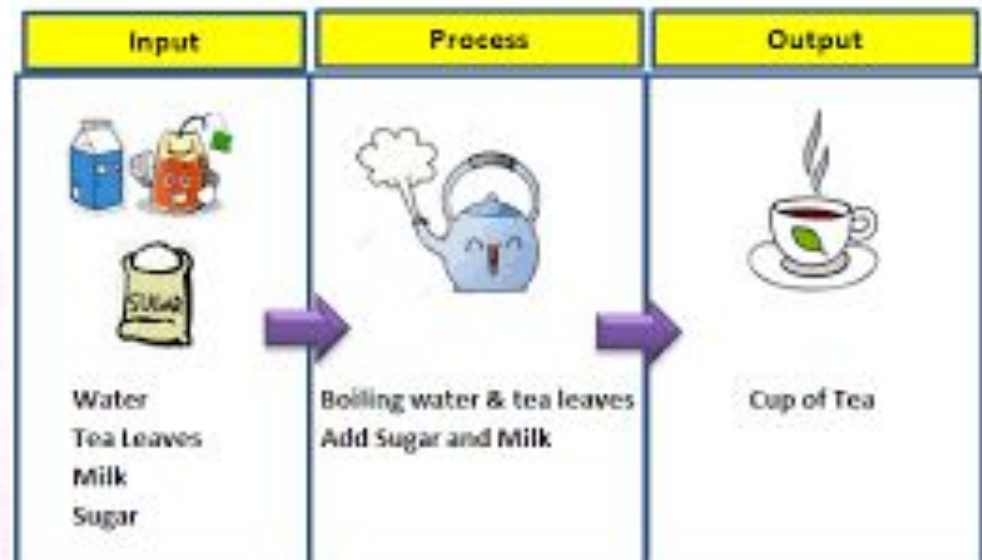
Syllabus/Contents

- Process Concept,
- Operations on Processes,
- Process Scheduling,
- Inter-process Communication, 5 Examples of IPC Systems
- Multithreading Models, Threading Issues,
- CPU Scheduling, Basic Concepts, Scheduling Criteria, Scheduling Algorithms, Multiple-Processor Scheduling, Thread Scheduling



Process

- Program under execution
- An instance of a program
- Schedulable/Dispatch unit (CPU)
- Unit of execution (CPU)





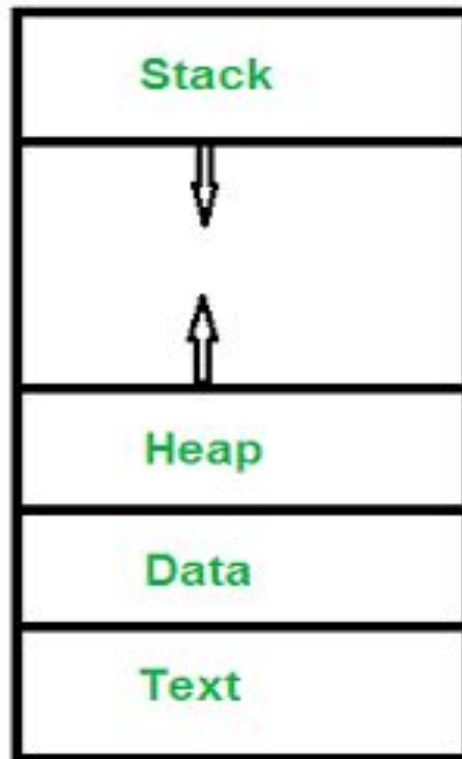
Process as a Data structure

- Definition (Same as last slide)
- Representation/Implementation
- Operations
- Attributes





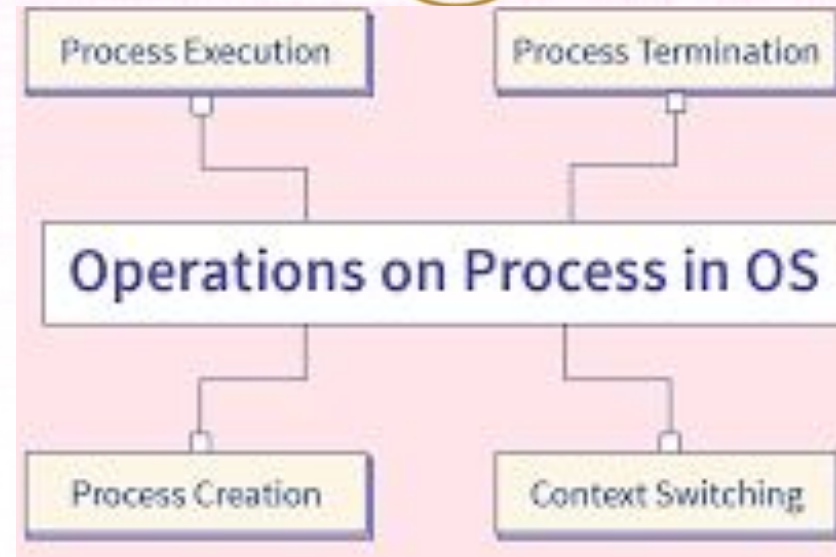
Process representation





Process operations

- Create (space allocation)
- Schedule/run
- Wait/Block
- Suspend/Resume
- Terminate (space deallocation)





Process Attributes in Process Control Block (PCB)

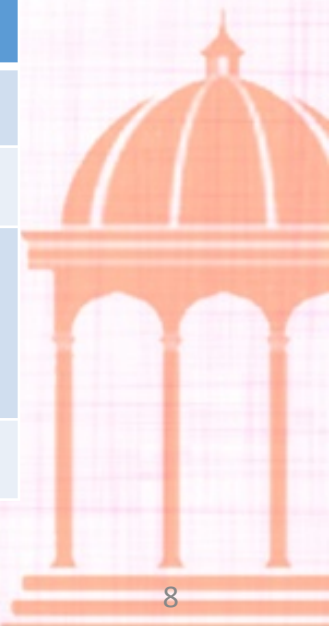




More about PCBs

- OS maintains 1 PCB for each process
- Contents of a PCB are also called collectively as context of a process

OS PCB1, PCB2, PCB3...PCBn
Process 1 (Stack, heap, code, data sections)
Process 2 (Stack, heap, code, data sections)
Process 3 (Stack, heap, code, data sections)
.
.
Process n (Stack, heap, code, data sections)





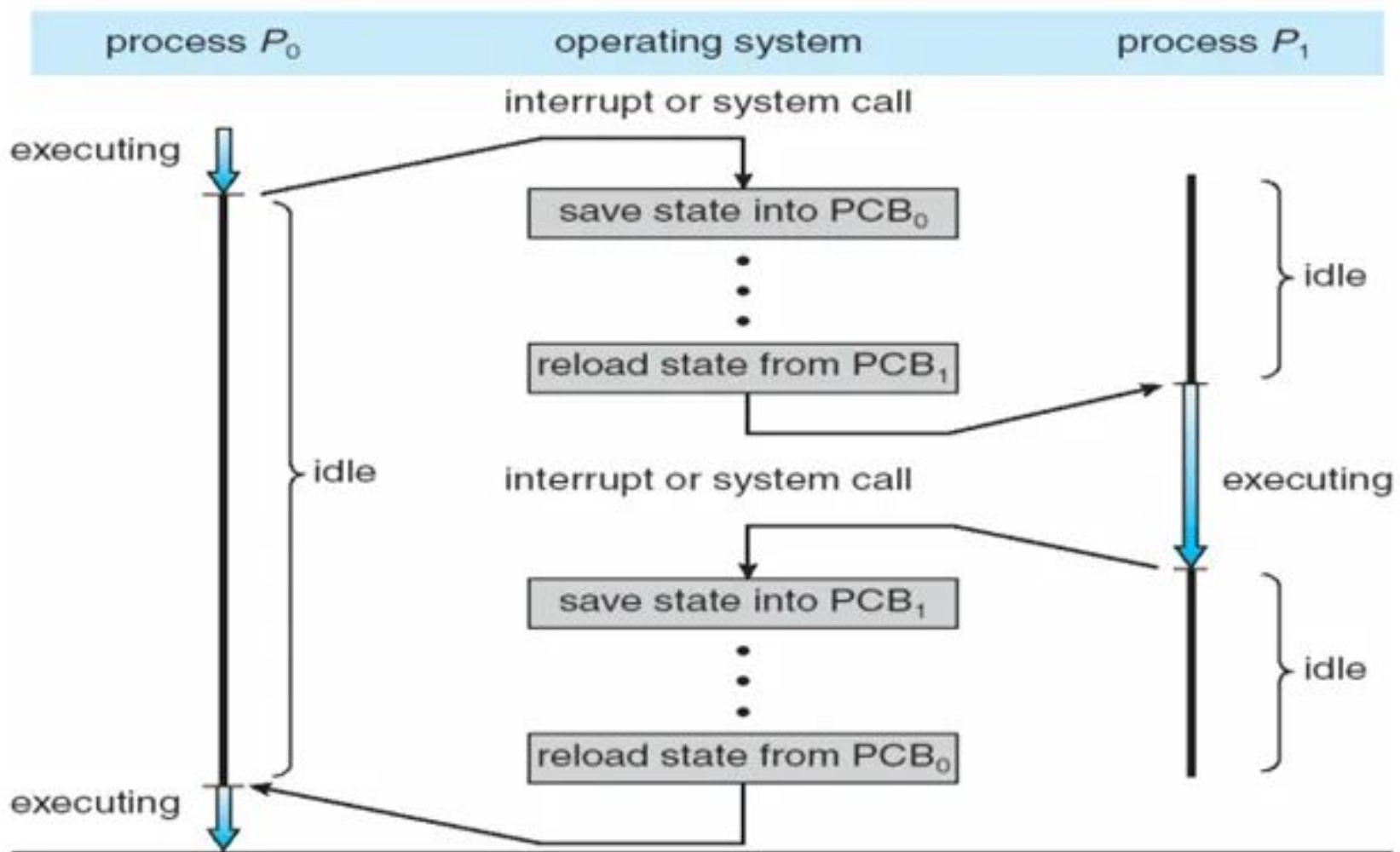
Context Switching in OS

- CPU uses registers to store current process execution state
- When stopped in between, have to save these values in PCB to resume from that point onwards
- Saving current running process's content(CPU register values) from CPU to PCB of the process and loading other process's context in CPU registers



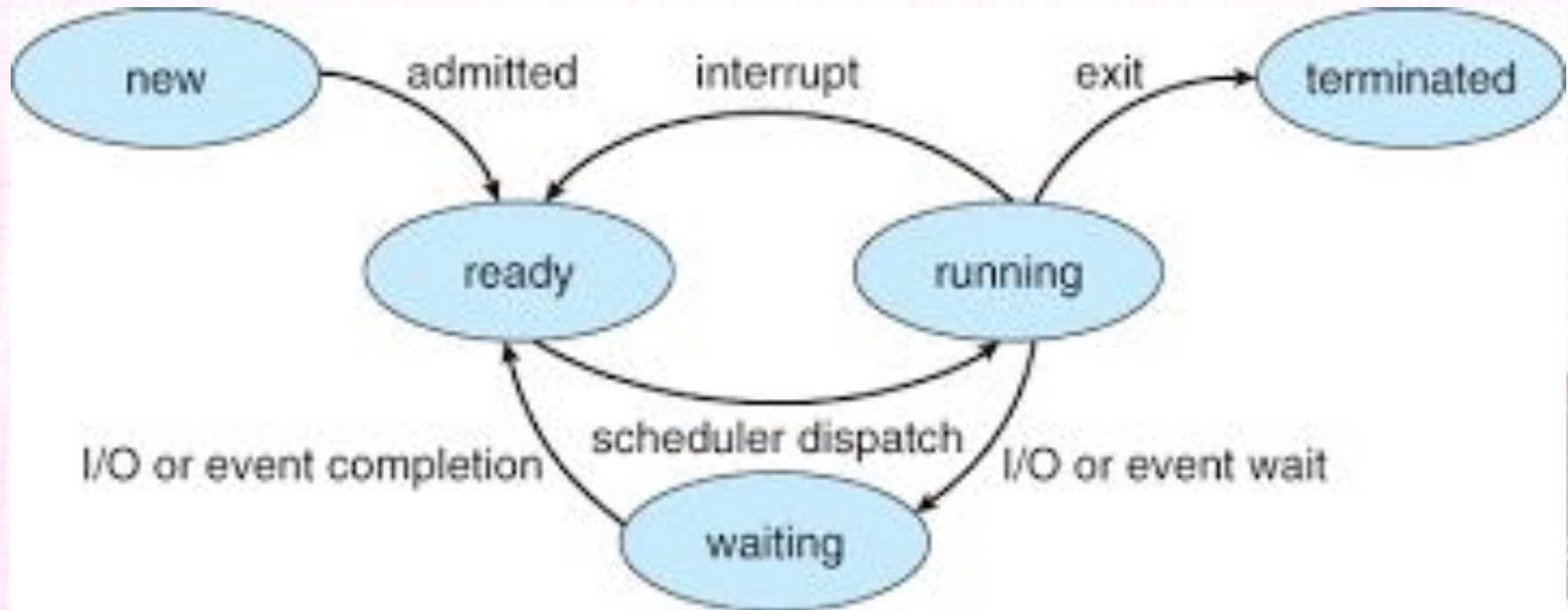


Example Context Switching



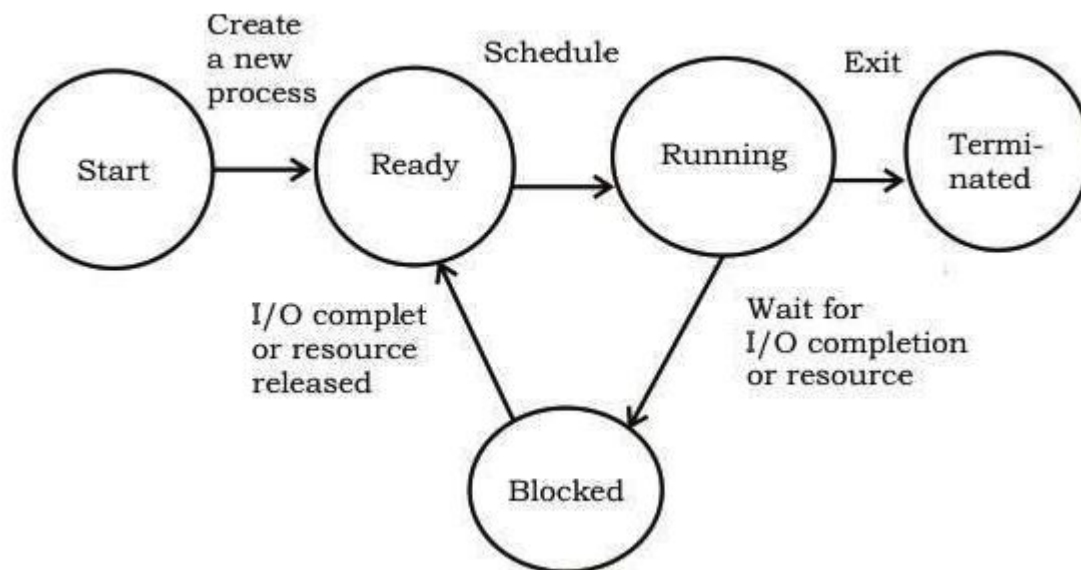


Preemptive Process States





Non-preemptive process states





Explanation

- New: All installed processes are here (SSD, HDD)
- Ready: Processes among the installed processes that are wanting to run on CPU (in MM)
- Running: Running on CPU (MM)
- Blocked: Waiting for any I/O or event (MM)
- Terminated: All completed processes are in this state (goes out from the MM)





Note

- When a new process is formed, OS makes the PCB
- In non preemptive process, no factor can force a running process to come out of CPU. Termination or going to waiting is only its wish.
- PCB will be there in running, ready and waiting state. In these three states, process will be in MM only





CPU scheduling

- Make a selection of ready processes to run on CPU.
- Needed for better performance
- Metric for performance may vary
- Multiple policies according to metric





Scheduling Queue

- Job queue: to keep all processes in new state
- Ready queue: to keep all processes in ready state
- Device queue: to keep the processes in blocked state





Scheduling Objectives

- Maximize throughput.
- Maximize number of users receiving acceptable response times.
- Be predictable.
- Balance resource use.
- Enforce Priorities.
- Give preference to processes holding key resources





CPU scheduling need

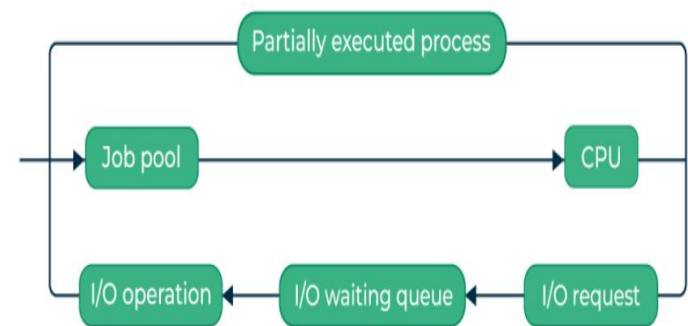
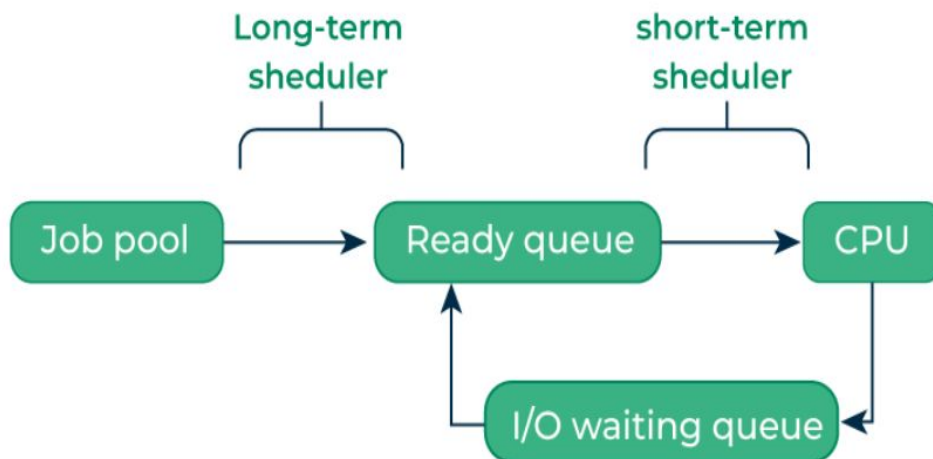
- Make a selection of ready processes to run on CPU.
- Needed for better performance
- Metric for performance may vary
- Multiple policies according to metric





Types of scheduler

- Long term scheduler: new to ready
- Short term scheduler: ready to running
- Mid term scheduler: swapping is done by this. Swapping leads to suspension





Comparison between scheduler.

S.N	Long Term	Short Term	Medium Term
1	It is job scheduler	It is CPU Scheduler	It is swapping
2	Speed is less than short term scheduler	Speed is very fast	Speed is in between both
3	It controls degree of multiprogramming	Less control over degree of multiprogramming	Reduce the degree of multiprogramming.
4	Absent or minimal in time sharing system.	Minimal in time sharing system.	Time sharing system use medium term scheduler.
5	It select processes from pool and load them into memory for execution.	It select from among the processes that are ready to execute.	Process can be reintroduced into memory and its execution can be continued.
6	Process state is (New to Ready)	Process state is (Ready to Running)	swapping
7	Select a good process, mix of I/O bound and CPU bound.	Select a new process for a CPU quite frequently.	



CPU Scheduling

- Types: Preemptive and non preemptive
- Assumption initially: No process has I/O requirement
- Some terminologies:
 - Arrival Time(AT): Process arrival time(admitted by LTS)
 - Burst/Service time(BT): burst time is the time, required the cpu to execute that job, it is in milli seconds.
 - Completion time (CT): Time when process completes execution
 - Turn around time (TAT): Time from arrival until completion ($TAT=CT-AT$)
 - Waiting time (WT): Waiting for CPU in ready queue ($WT=TAT-BT$)
 - Response time: Time from arrival to the first response on CPU
 - Scheduling length: $\max(CT)-\min(AT)$
 - Throughput: No of processes executed per unit time



Scheduling Criteria

- CPU Utilization
- Throughput
- Turnaround time
- Waiting Time
- Response Time





CPU SCHEDULING ALGORITHMS:

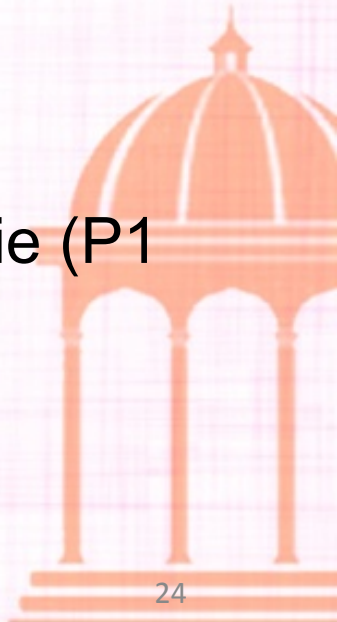
- FCFS (First come first serve) – Non preemptive
- SJF (Shortest Job first) – Non preemptive
- Priority Scheduling
- Round Robin - Preemptive
- Others





FCFS (First Come First Serve)

- The process that request the CPU first is holds the cpu first.
- If a process request the cpu then it is loaded into the ready queue
- Mode: Non preemptive
- Tie breaker: Smallest ID first of the processes at tie (P1 and P2, P1 will execute first)





FCFS Example no. 1

Process Id	Arrival time	Burst time
P1	3	4
P2	5	3
P3	0	2
P4	5	1
P5	4	3

calculate the average waiting time and average turn around time.





Solution-

Gantt Chart-



Gantt Chart

Here, black box represents the idle time of CPU.

- Turn Around time = Exit time – Arrival time
- Waiting time = Turn Around time – Burst time



rocess Id	Exit time	Turn Around time	Waiting time
P1	7	$7 - 3 = 4$	$4 - 4 = 0$
P2	13	$13 - 5 = 8$	$8 - 3 = 5$
P3	2	$2 - 0 = 2$	$2 - 2 = 0$
P4	14	$14 - 5 = 9$	$9 - 1 = 8$
P5	10	$10 - 4 = 6$	$6 - 3 = 3$



- Average Turn Around time = $(4 + 8 + 2 + 9 + 6) / 5 = 29 / 5 = 5.8$ unit
- Average waiting time = $(0 + 5 + 0 + 8 + 3) / 5 = 16 / 5 = 3.2$ unit





Pros and Cons

- Advantages
 - Very easy to implement
 - No complex logic
 - No starvation
- Disadvantages
 - No option of preemption
 - Convoy effect





Convoy effect

- Only FCFS suffers
- If a slow process (large service time) is scheduled first, then it makes entire system slow





SJF

- Criteria: min BT first
- Mode: Non preemptive
- Tie breaker: FCFS



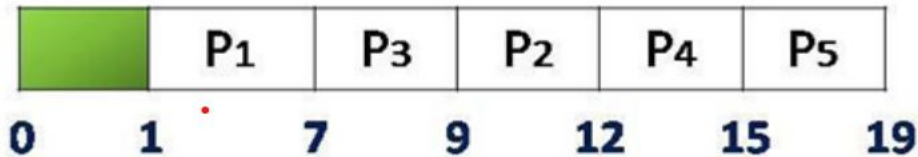


Example

Process No.	Arrival Time (AT)	Burst Time (BT)
1	1	6
2	2	3
3	3	2
4	4	3
5	5	4



Gantt chart:



TAT = CT - AT and WT = TAT - BT

Process No.	AT	BT	CT	TAT	WT
1	1	6	7	$(7 - 1) = 6$	$(6 - 6) = 0$
2	2	3	12	$(12 - 2) = 10$	$(10 - 3) = 7$
3	3	2	9	$(9 - 3) = 6$	$(6 - 2) = 4$
4	4	3	15	$(15 - 4) = 11$	$(11 - 3) = 8$
5	5	4	19	$(19 - 5) = 14$	$(14 - 4) = 10$



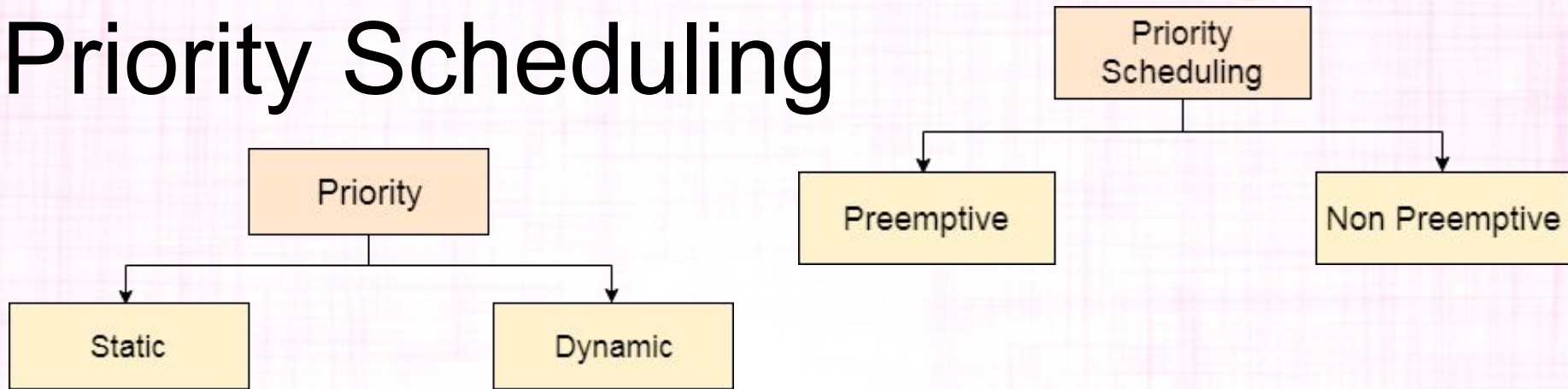
More about SJF

- Advantages:
 - Min waiting time among non preemptive scheduling
 - Better throughput in continuous execution
 - No convoy effect
- Disadvantages
 - No practical implementation because burst time cannot be known in advance
 - No option of preemption
 - Longer processes may suffer from starvation
 - No fairness





Priority Scheduling



Process ID	Priority	Arrival Time	Burst Time
1	2	0	3
2	6	2	5
3	3	1	4
4	5	4	2
5	7	6	9
6	4	5	4
7	10	7	10





P1	P3	P6	P4	P2	P5	P7
0	3	7	11	13	18	27
						37

1. Turn Around **Time** = **Completion** Time - Arrival Time

2. Waiting **Time** = **Turn** Around Time - Burst Time

Process Id	Priority	Arrival Time	Burst Time	Completion Time	Turnaround Time	Waiting Time	Response Time
1	2	0	3	3	3	0	0
2	6	2	5	18	16	11	13
3	3	1	4	7	6	2	3
4	5	4	2	13	9	7	11
5	7	6	9	27	21	12	18
6	4	5	4	11	6	2	7
7	10	7	10	37	30	18	27



Avg Waiting Time = $(0+11+2+7+12+2+18)/7 = 52/7$ units

Preemptive Priority Scheduling

Process Id	Priority	Arrival Time	Burst Time
1	2(L)	0	1
2	6	1	7
3	3	2	3
4	5	3	6
5	4	4	5
6	10(H)	5	15
7	9	15	8

P1	P2	P3	P5	P4	P2	P7	P6	
0	1	2	5	10	16	22	30	45





Process Id	Priority	Arrival Time	Burst Time	Completion Time	Turn around Time	Waiting Time
1	2	0	1	1	1	0
2	6	1	7	22	21	14
3	3	2	3	5	3	0
4	5	3	6	16	13	7
5	4	4	5	10	6	1
6	10	5	15	45	40	25
7	9	6	8	30	24	16



Round Robin Scheduling

- Criteria: Arrival time + (Q)Quantum time/timeslice
- Mode: Preemptive
- Quantum time/time slice: Max amount of time for which a process runs on CPU before preemption



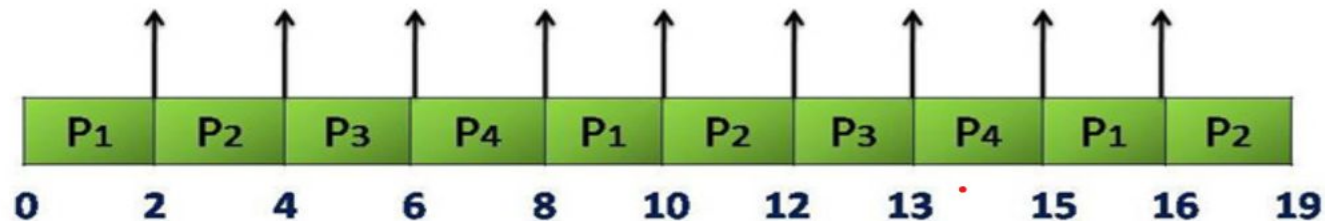


Example 1: TQ = 2 (Here)

Process No.	Arrival Time (AT)	Burst Time (BT)
P ₁	0	5
P ₂	1	7
P ₃	2	3
P ₄	3	4

Gantt Chart

Context Switching





Now we calculate **Turn Around Time (TAT)** and **Waiting Time (WT)** using the following formula:

$$\text{TAT} = \text{CT} - \text{AT}, \text{ WT} = \text{TAT} - \text{BT}$$

And

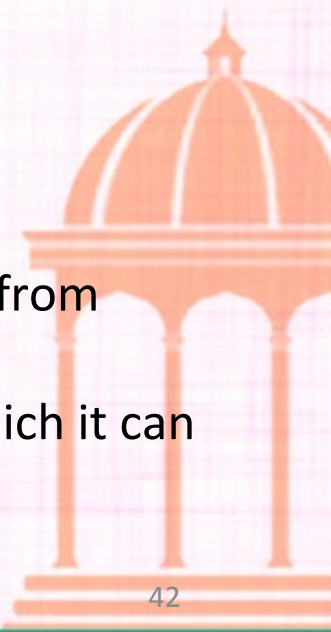
$$\text{Response Time (RT)} = \text{FR (First Response)} - \text{AR (Arrival Time)}$$

P_No.	AT	BT	CT	TAT	WT	First Response (FR)	Response Time (RT)
P1	0	5 3 1	16	$(16 - 0) = 16$	$(16 - 5) = 11$	0	$(0 - 0) = 0$
P2	1	7 5 3	19	$(19 - 1) = 18$	$(18 - 7) = 11$	2	$(2 - 1) = 1$
P3	2	3 2 1	13	$(13 - 2) = 11$	$(11 - 3) = 8$	4	$(4 - 2) = 2$
P4	3	4 2 2	15	$(15 - 3) = 12$	$(12 - 4) = 8$	6	$(6 - 3) = 3$



Multilevel Queue

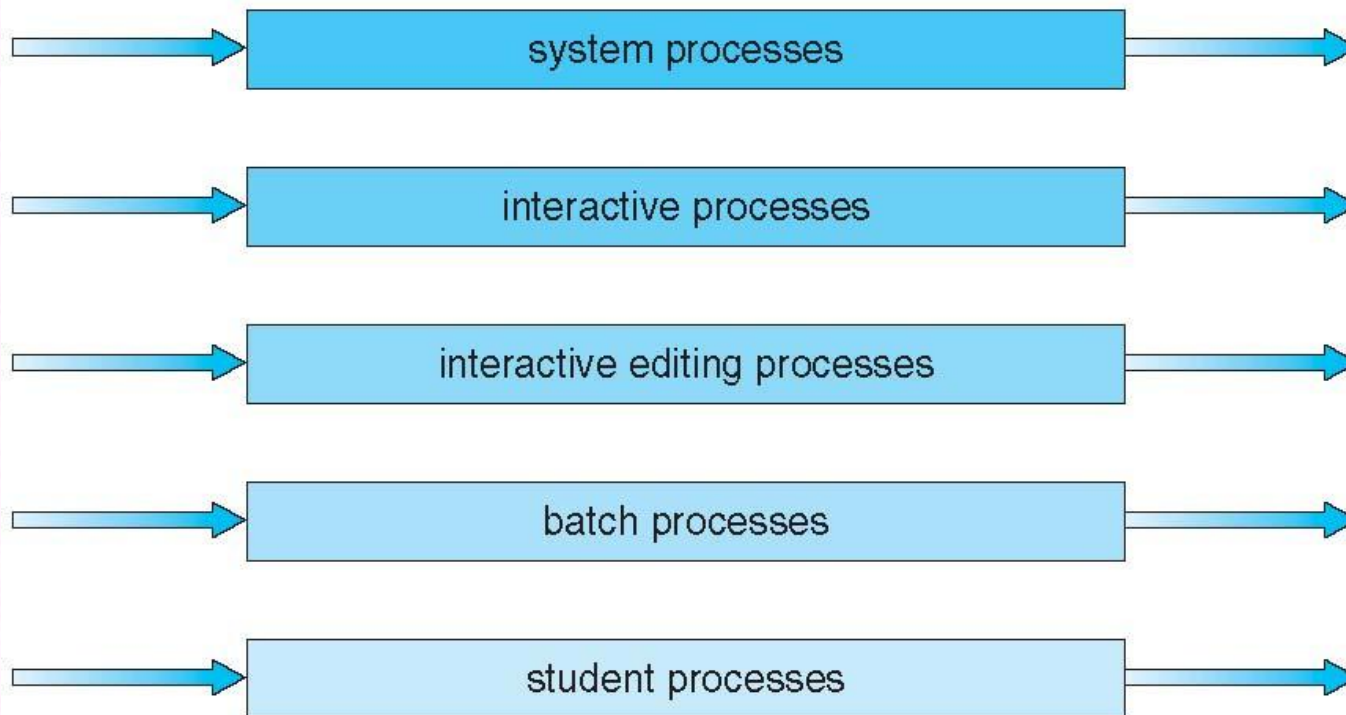
- Ready queue is partitioned into separate queues, eg:
 - **foreground** (interactive)
 - **background** (batch)
- Process permanently in a given queue
- Each queue has its own scheduling algorithm:
 - foreground – RR
 - background – FCFS
- Scheduling must be done between the queues:
 - Fixed priority scheduling; (i.e., serve all from foreground then from background). Possibility of starvation.
 - Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR
 - 20% to background in FCFS





Multilevel Queue Scheduling

highest priority



lowest priority





Multilevel Feedback Queue

- A process can move between the various queues; aging can be implemented this way
- Multilevel-feedback-queue scheduler defined by the following parameters:
 - number of queues
 - scheduling algorithms for each queue
 - method used to determine when to upgrade a process
 - method used to determine when to demote a process
 - method used to determine which queue a process will enter when that process needs service





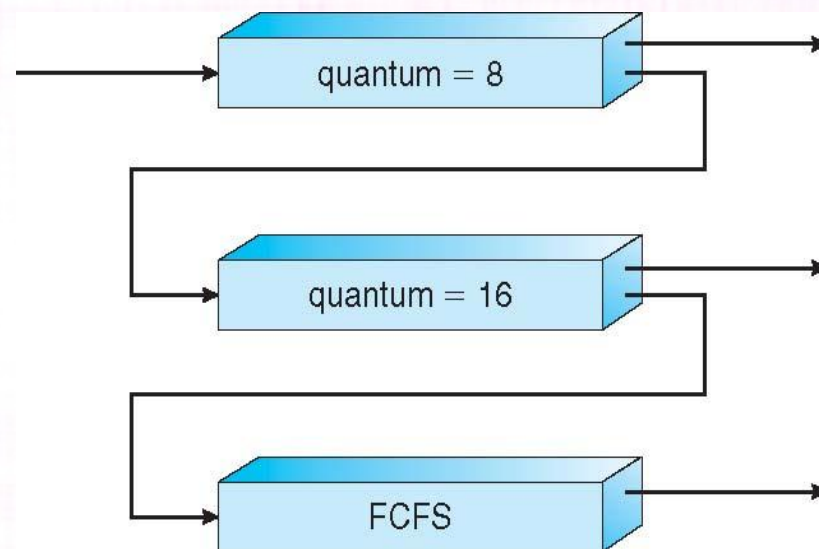
Example of Multilevel Feedback Queue

- Three queues:

- Q_0 – RR with time quantum 8 milliseconds
- Q_1 – RR time quantum 16 milliseconds
- Q_2 – FCFS

- Scheduling

- A new job enters queue Q_0 which is served FCFS
 - When it gains CPU, job receives 8 milliseconds
 - If it does not finish in 8 milliseconds, job is moved to queue Q_1
- At Q_1 job is again served FCFS and receives 16 additional milliseconds
 - If it still does not complete, it is preempted and moved to queue Q_2





Thread Scheduling

- Distinction between user-level and kernel-level threads
- When threads supported, threads scheduled, not processes
- Many-to-one and many-to-many models, thread library schedules user-level threads to run on LWP
 - Known as **process-contention scope (PCS)** since scheduling competition is within the process
 - Typically done via priority set by programmer
- Kernel thread scheduled onto available CPU is **system-contention scope (SCS)** – competition among all threads in system



Multiple-Processor Scheduling

- CPU scheduling more complex when multiple CPUs are available
- **Homogeneous processors** within a multiprocessor
- **Asymmetric multiprocessing** – only one processor accesses the system data structures, alleviating the need for data sharing
- **Symmetric multiprocessing (SMP)** – each processor is self-scheduling, all processes in common ready queue, or each has its own private queue of ready processes
 - Currently, most common
- **Processor affinity** – process has affinity for processor on which it is currently running
 - **soft affinity**
 - **hard affinity**
 - Variations including **processor sets**



Multiple-Processor Scheduling – Load Balancing

- If SMP, need to keep all CPUs loaded for efficiency
- **Load balancing** attempts to keep workload evenly distributed
- **Push migration** – periodic task checks load on each processor, and if found pushes task from overloaded CPU to other CPUs
- **Pull migration** – idle processors pulls waiting task from busy processor





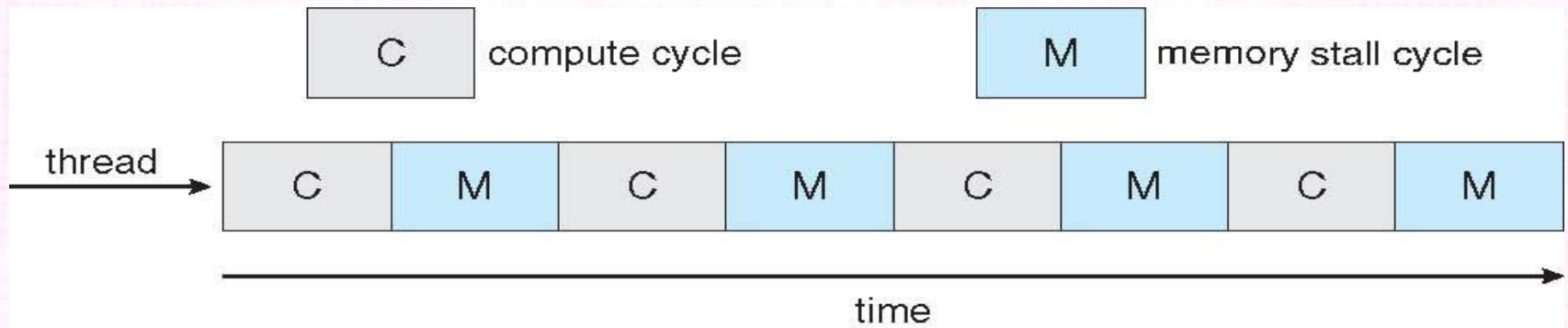
Multicore Processors

- Recent trend to place multiple processor cores on same physical chip
- Faster and consumes less power
- Multiple threads per core also growing
 - Takes advantage of memory stall to make progress on another thread while memory retrieve happens





Multithreaded Multicore System





- **Inter Process Communication (IPC)**
 - Inter process communication in OS is the way by which multiple processes can communicate with each other.
1. Independent Process.
 2. Co-operating Process.

Independent Process	Co-operating Process
1. A process is independent if it cannot affect or be affected by the other processes executing in the system	1. A process is co-operating if it can affect or be affected by the other processes executing in the system.
2. Any process that does not share data with any other process is IP	2. Any process that share data with any other process is CP



- **Inter Process Communication (IPC)**
- IPC facility provides two operations:
 1. send(message) – message size fixed or variable
 2. receive(message)
- If P and Q wish to communicate, they need to:
 1. establish a communication link between them
 2. exchange messages via send/receive





- Inter Process Communication (IPC)**

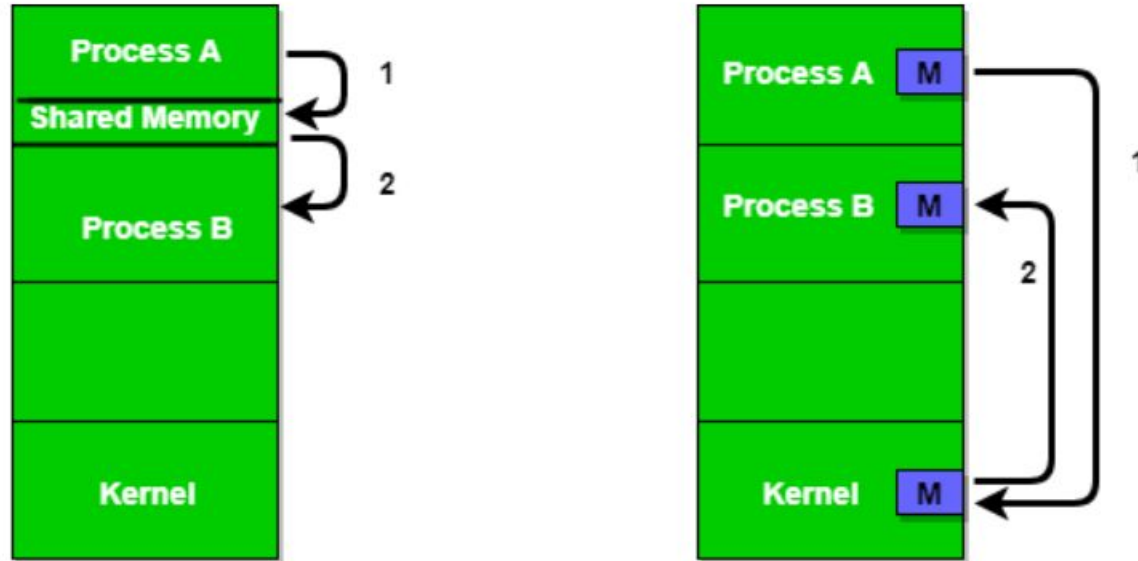


Figure 1 - Shared Memory and Message Passing





The inter-process communication examples include the following.

- Posix uses the shared memory technique.
- Windows XP uses message passing technique.
- Mach uses the message passing technique.
- Pipes.
- Server.





Advantages

- It permits one application to manage another application
- Allows data communication through different processes to utilize semaphores, segments & other techniques to share data & memory.
- helps in transferring efficient messages in between processes.
- avoid synchronization & named pipes blocking issues by sending a message.





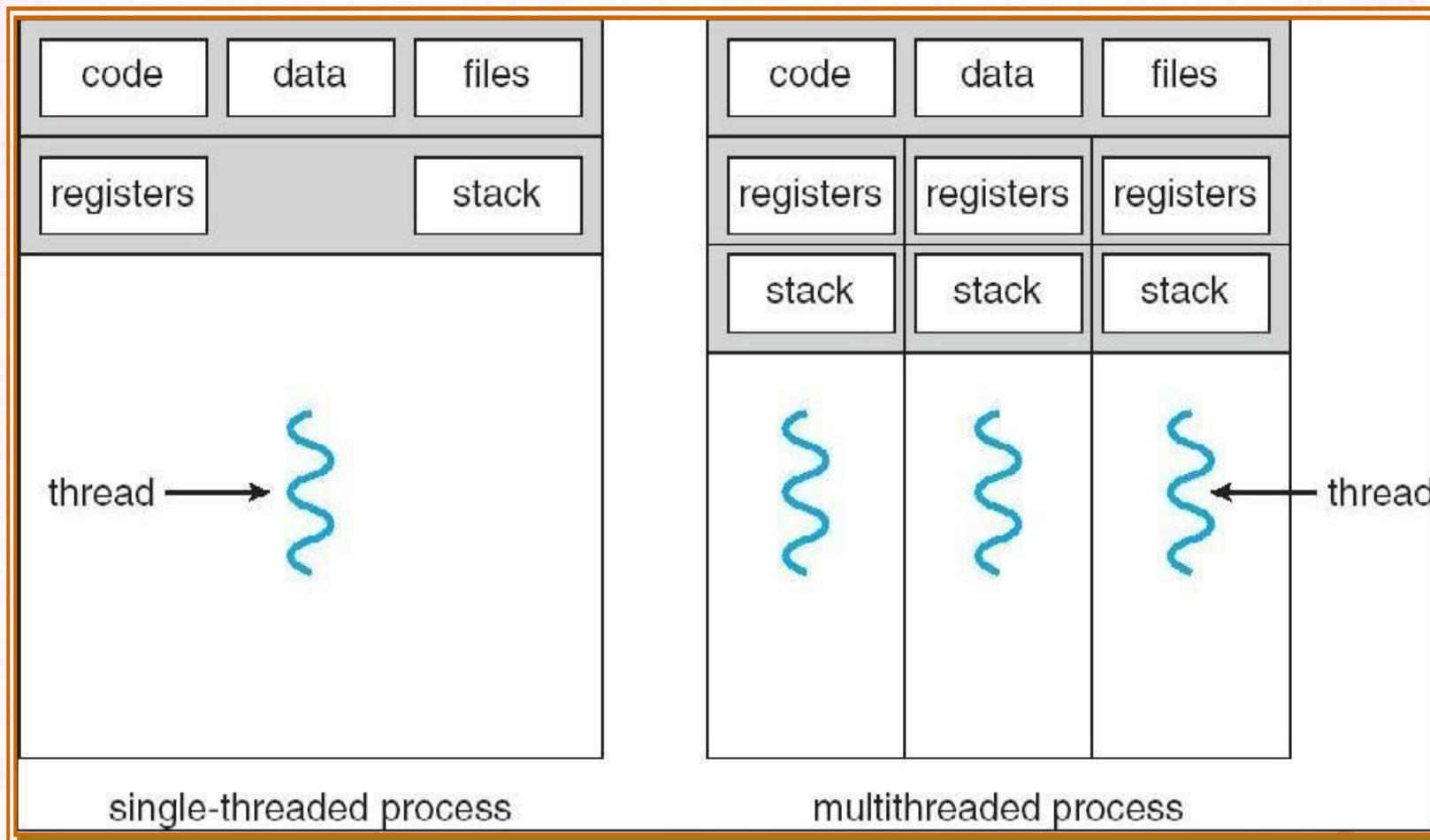
Disadvantages

- Similar to the pipe, every data block includes the highest limit of length.
- Complete system data length for all blocks integrated within the queue also includes an upper limit.
- All the processes or programs that utilize the model of shared memory need to check that they are not writing to a similar memory location.
- A shared memory model may form issues like synchronization & protection of memory that require to be addressed.
- As compared to straight function calls, it is slow.



THREADS

- A Thread is a flow of execution through the process code, with its own program counter that keeps track of which instruction to execute next, system registers .
- It is also called as light weight process
- A thread is a basic unit of CPU utilization; it comprises a thread ID, a program counter, a register set, and a stack.
- It shares with other threads belonging to the same process its code section, data section, and other operating-system resources, such as open files and signals.
- THREADS Overview
- A word processor may have a thread for displaying graphics, another thread for responding to keystrokes from the user, and a third thread
- for performing spelling and grammar checking in the background.

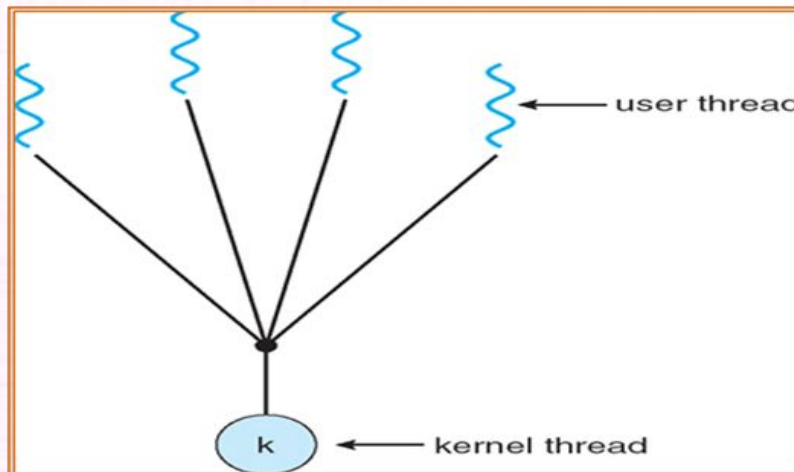




Multithreading Models

- Many-to-One
- One-to-One
- Many-to-Many

1. Many-to-One



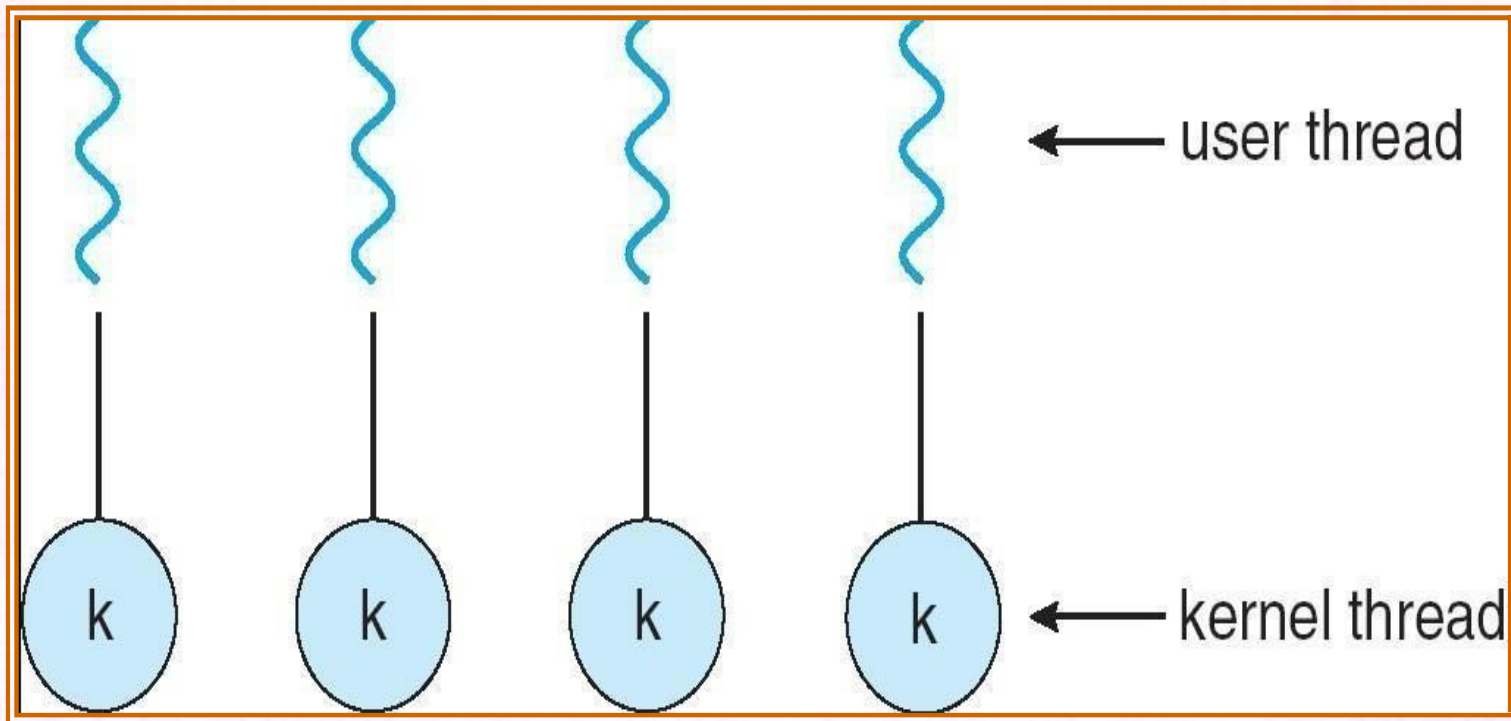
Many-to-One Model





Multithreading Models

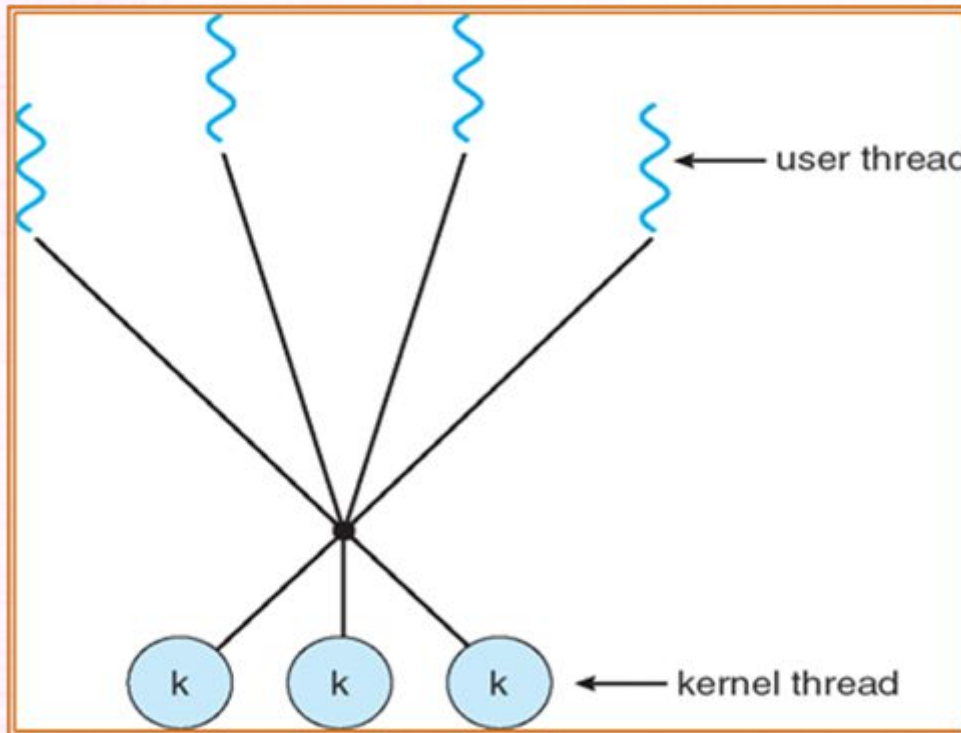
2. One-to-One





Multithreading Models

3. Many to Many Model



Many-to-Many Model





Threading Issues:

- Semantics of **fork()** and **exec()** system calls
- Thread cancellation
- Signal handling
- Thread pools
- Thread specific data
- Scheduler activations





Threading Issues in the Different Environment

Windows XP Threads

Each thread contains

- A thread id
- Register set
- Separate user and kernel stacks
- Private data storage area
- The register set, stacks, and private storage area are known as the **context** of the threads
- The primary data structures of a thread include:
 - ETHREAD (executive thread block)
 - KTHREAD (kernel thread block)
 - TEB (thread environment block).

Linux Thread:

Linux refers to them as *tasks* rather than *threads*

- Thread creation is done through clone() system call
- clone() allows a child task to share the address space of the parent task (process)

Java Threads:

Java threads are managed by the JVM Java threads may be created by:

- Extending Thread class
- Implementing the Runnable interface





THANK
YOU



MIT School of Computing

Department of Computer Science & Engineering



MIT-ADT
UNIVERSITY
PUNE, INDIA





MIT School of Computing

Department of Computer Science & Engineering



MIT-ADT
UNIVERSITY
PUNE, INDIA

